

Atividade de JavaScript

Desenvolvimento de Sistemas Web

Professor: Irlan Arley Targino Moreira | Aluno: Kellyson Medeiros

Data de Entrega: 4 de setembro

Questão 1. Conversão de duração em segundos para horas, minutos e segundos

```
function converterSegundos(totalSegundos) {  
  const horas = Math.floor(totalSegundos / 3600);  
  const minutos = Math.floor((totalSegundos % 3600) / 60);  
  const segundos = totalSegundos % 60;  
  return `${horas}h ${minutos}min ${segundos}s`;  
}  
  
console.log(converterSegundos(3665)); // 1h 1min 5s
```

Questão 2. Conversão de Celsius para Fahrenheit e Kelvin

```
function converterTemperatura(celsius) {  
  const fahrenheit = (celsius * 9) / 5 + 32;  
  const kelvin = celsius + 273.15;  
  return {  
    celsius,  
    fahrenheit: Number(fahrenheit.toFixed(2)),  
    kelvin: Number(kelvin.toFixed(2))  
  };  
}  
  
console.log(converterTemperatura(25));  
// { celsius: 25, fahrenheit: 77, kelvin: 298.15 }
```

Questão 3. Cálculo de pedido com desconto progressivo e template literals

```
function resumoPedido(preco, quantidade) {  
  const subtotal = preco * quantidade;  
  let percentualDesconto = 0;  
  
  if (subtotal > 300) {  
    percentualDesconto = 0.10;  
  } else if (subtotal > 100) {  
    percentualDesconto = 0.05;  
  }  
  
  const valorDesconto = subtotal * percentualDesconto;  
  const total = subtotal - valorDesconto;
```

```

        return `--- Resumo do Pedido ---
Preço unitário: R$ ${preco.toFixed(2)}
Quantidade: ${quantidade}
Subtotal: R$ ${subtotal.toFixed(2)}
Desconto (${percentualDesconto * 100}%): R$ ${valorDesconto.toFixed(2)}
Total a pagar: R$ ${total.toFixed(2)}`;
    }

    console.log(resumoPedido(50, 4));

```

Questão 4. Idade, situação eleitoral e serviço militar

```

function verificarSituacao(anoNascimento) {
    const anoAtual = new Date().getFullYear();
    const idade = anoAtual - anoNascimento;

    const podeVotar = idade >= 16;
    const votoObrigatorio = idade >= 18 && idade < 70;
    const isentoServicoMilitar = idade < 18;

    return { idade, podeVotar, votoObrigatorio, isentoServicoMilitar };
}

console.log(verificarSituacao(2008));
// { idade: 18, podeVotar: true, votoObrigatorio: true, isentoServicoMilitar: false }

```

Questão 5. Calculadora com switch/case, divisão por zero e operador inválido

```

function calculadora(a, b, operador) {
    switch (operador) {
        case '+':
            return a + b;
        case '-':
            return a - b;
        case '*':
            return a * b;
        case '/':
            if (b === 0) {
                return "Erro: divisão por zero";
            }
            return a / b;
        default:
            return "Erro: operador inválido";
    }
}

console.log(calculadora(10, 2, '+')); // 12
console.log(calculadora(10, 0, '/')); // Erro: divisão por zero

```

```
console.log(calculadora(10, 2, '%')); // Erro: operador inválido
```

Questão 6. Classificação de IMC

```
function classificarIMC(peso, altura) {
  const imc = peso / (altura * altura);

  if (imc < 18.5) {
    return "Abaixo do peso";
  } else if (imc < 25) {
    return "Normal";
  } else if (imc < 30) {
    return "Sobrepeso";
  } else {
    return "Obesidade";
  }
}

console.log(classificarIMC(70, 1.75)); // Normal
```

Questão 7. Pedra, Papel e Tesoura sem comparações repetitivas

```
function pedraPapelTesoura(jogador1, jogador2) {
  const j1 = jogador1.toLowerCase();
  const j2 = jogador2.toLowerCase();

  if (j1 === j2) {
    return "Empate";
  }

  const vitorias = {
    pedra: 'tesoura',
    papel: 'pedra',
    tesoura: 'papel'
  };

  if (vitorias[j1] === j2) {
    return "Jogador 1 venceu";
  }

  return "Jogador 2 venceu";
}

console.log(pedraPapelTesoura("pedra", "tesoura")); // Jogador 1 venceu
console.log(pedraPapelTesoura("papel", "papel")); // Empate
```

Questão 8. Verificação de Palíndromo

```
function ehPalindromo(texto) {
  const limpo = texto.toLowerCase().replace(/\s+/g, '');
  const invertido = limpo.split('').reverse().join('');
  return limpo === invertido;
}

console.log(ehPalindromo("A base do teto desaba")); // true
console.log(ehPalindromo("javascript"));           // false
```

Questão 9. Estatísticas de notas (média, maior e menor)

```
function analisarNotas(notas) {
  if (!notas || notas.length === 0) return null;

  const soma = notas.reduce((acc, nota) => acc + nota, 0);
  const media = soma / notas.length;
  const maior = Math.max(...notas);
  const menor = Math.min(...notas);

  return {
    media: Number(media.toFixed(2)),
    maior,
    menor
  };
}

console.log(analisarNotas([7.5, 8.0, 9.5, 6.0, 10.0]));
// { media: 8.2, maior: 10, menor: 6 }
```

Questão 10. Cálculo do n-ésimo termo de Fibonacci

```
function fibonacci(n) {
  if (n <= 0) return 0;
  if (n === 1) return 1;

  let anterior = 0;
  let atual = 1;

  for (let i = 2; i <= n; i++) {
    const proximo = anterior + atual;
    anterior = atual;
    atual = proximo;
  }

  return atual;
}

console.log(fibonacci(7)); // 13
```

Questão 11. Validação de regras de senha

```
function validarSenha(senha) {
  const pendencias = [];

  if (senha.length < 8) {
    pendencias.push("Mínimo de 8 caracteres");
  }
  if (!/[A-Z]/.test(senha)) {
    pendencias.push("Ao menos uma letra maiúscula");
  }
  if (!/[0-9]/.test(senha)) {
    pendencias.push("Ao menos um número");
  }

  return pendencias;
}

console.log(validarSenha("abc"));
// ['Mínimo de 8 caracteres', 'Ao menos uma letra maiúscula', 'Ao menos um número']
console.log(validarSenha("Senha123")); // []
```

Questão 12. Estoque de produtos (total, mais caro e abaixo do mínimo)

```
function calcularValorTotalEstoque(produtos) {
  return produtos.reduce((total, p) => total + p.preco * p.quantidade, 0);
}

function encontrarProdutoMaisCaro(produtos) {
  return produtos.reduce((maisCaro, p) => (p.preco > maisCaro.preco ? p : maisCaro),
produtos[0]);
}

function listarAbaixoDoMinimo(produtos, minimo) {
  return produtos.filter(p => p.quantidade < minimo);
}

const produtos = [
  { nome: "Teclado", preco: 250, quantidade: 5 },
  { nome: "Mouse", preco: 120, quantidade: 2 },
  { nome: "Monitor", preco: 900, quantidade: 1 }
];

console.log("Total:", calcularValorTotalEstoque(produtos)); // 2450
console.log("Mais caro:", encontrarProdutoMaisCaro(produtos)); // Monitor
console.log("Abaixo de 3 unidades:", listarAbaixoDoMinimo(produtos, 3));
```

Questão 13. Agenda de contatos e função de valores únicos

```
let agenda = [
```

```

    { nome: "Carlos", telefone: "9999-1111", categoria: "Trabalho" },
    { nome: "Mariana", telefone: "9888-2222", categoria: "Amigos" }
  ];

  function adicionarContato(agenda, nome, telefone, categoria) {
    agenda.push({ nome, telefone, categoria });
    return agenda;
  }

  function removerContatoPorNome(agenda, nome) {
    const index = agenda.findIndex(c => c.nome.toLowerCase() === nome.toLowerCase());
    if (index !== -1) {
      agenda.splice(index, 1);
    }
    return agenda;
  }

  function listarPorCategoria(agenda, categoria) {
    return agenda.filter(c => c.categoria.toLowerCase() === categoria.toLowerCase());
  }

  // Complemento: função para retornar apenas valores únicos
  function obterValoresUnicos(array) {
    return [...new Set(array)];
  }

  console.log(listarPorCategoria(agenda, "Trabalho"));
  console.log(obterValoresUnicos([1, 2, 2, 3, 4, 4, 5])); // [1, 2, 3, 4, 5]

```

Questão 14. Classe Produto com métodos de desconto e disponibilidade

```

class Produto {
  constructor(nome, preco, quantidade) {
    this.nome = nome;
    this.preco = preco;
    this.quantidade = quantidade;
  }

  aplicarDesconto(percentual) {
    this.preco = this.preco * (1 - percentual / 100);
  }

  estaDisponivel() {
    return this.quantidade > 0;
  }
}

const p1 = new Produto("Notebook", 3500, 4);
const p2 = new Produto("Fone Bluetooth", 150, 0);
const p3 = new Produto("Webcam", 200, 8);

```

```
p1.aplicarDesconto(10);

console.log(p1.nome, "- R$", p1.preco, "| Disponível:", p1.estaDisponivel());
console.log(p2.nome, "- R$", p2.preco, "| Disponível:", p2.estaDisponivel());
console.log(p3.nome, "- R$", p3.preco, "| Disponível:", p3.estaDisponivel());
```

Questão 15. Classe ContaBancaria com operações de depósito, saque e extrato

```
class ContaBancaria {
  constructor(titular, saldo = 0) {
    this.titular = titular;
    this.saldo = saldo;
  }

  depositar(valor) {
    if (valor > 0) {
      this.saldo += valor;
      return true;
    }
    return false;
  }

  sacar(valor) {
    if (valor > 0 && valor <= this.saldo) {
      this.saldo -= valor;
      return true;
    }
    return false;
  }

  extrato() {
    return `Titular: ${this.titular} | Saldo: R$ ${this.saldo.toFixed(2)}`;
  }
}

const conta1 = new ContaBancaria("Kellyson", 500);
const conta2 = new ContaBancaria("Maria", 200);

conta1.depositar(150);
conta1.sacar(100);
conta2.depositar(50);
conta2.sacar(400); // Saldo insuficiente, não debita

console.log(conta1.extrato());
console.log(conta2.extrato());
```

Questão 16. Classe Retangulo com cálculo de área e perímetro

```
class Retangulo {
```

```

    constructor(base, altura) {
        this.base = base;
        this.altura = altura;
    }

    calcularArea() {
        return this.base * this.altura;
    }

    calcularPerimetro() {
        return 2 * (this.base + this.altura);
    }
}

const ret = new Retangulo(10, 5);
console.log("Área:", ret.calcularArea());           // 50
console.log("Perímetro:", ret.calcularPerimetro()); // 30

```

Questão 17. Reescrita do exercício de Estoque usando Classe e Comparação

```

class Estoque {
    constructor(produtosIniciais = []) {
        this.produtos = produtosIniciais;
    }

    adicionarProduto(produto) {
        this.produtos.push(produto);
    }

    calcularValorTotal() {
        return this.produtos.reduce((total, p) => total + p.preco * p.quantidade, 0);
    }

    obterProdutoMaisCaro() {
        if (this.produtos.length === 0) return null;
        return this.produtos.reduce((maisCaro, p) => (p.preco > maisCaro.preco ? p : maisCaro), this.produtos[0]);
    }

    listarAbaixoDoMinimo(minimo) {
        return this.produtos.filter(p => p.quantidade < minimo);
    }
}

const estoque = new Estoque([
    { nome: "Teclado", preco: 250, quantidade: 5 },
    { nome: "Mouse", preco: 120, quantidade: 2 }
]);

console.log("Total em estoque:", estoque.calcularValorTotal());

```

Comparação entre as duas versões:

- Versão com Array de Objetos e Funções: Os dados ficam separados da lógica de negócio. A lista de produtos precisa ser repassada como argumento a cada chamada de função e fica exposta no escopo, tornando o código mais suscetível a mutações acidentais e difícil de reutilizar.
- Versão com Classe: Centraliza o estado (a lista de produtos) e todas as operações (métodos) dentro da mesma estrutura. Isso gera encapsulamento, melhora a legibilidade, evita passagem redundante de parâmetros e torna a manutenção e extensão do sistema muito mais organizada e segura.